

Hooks y su uso en Cypress

Hook	Descripción	Uso Común	Ejemplo
<code>before()</code>	Se ejecuta una vez antes de todos los tests en un bloque de test.	Inicializar datos o configuraciones globales para la suite de tests.	<pre>before(() => { // Configuración global cy.login(); });</pre>
<code>beforeEach()</code>	Se ejecuta antes de cada test individual.	Preparar el estado inicial o realizar tareas repetitivas antes de cada prueba.	<pre>beforeEach(() => { cy.visit('/home'); });</pre>
<code>after()</code>	Se ejecuta una vez después de todos los tests en un bloque de test.	Realizar limpieza global o restaurar el estado después de todos los tests.	<pre>after(() => { // Tareas de limpieza global cy.clearCookies(); });</pre>
<code>afterEach()</code>	Se ejecuta después de cada test individual.	Restaurar o limpiar estados después de cada prueba.	<pre>afterEach(() => { cy.clearLocalStorage(); });</pre> Activar Windows Ve a Configuración para activar Windows

Explicación de los hooks:

- `before()`: Ideal para realizar configuraciones o autenticaciones que solo se necesitan una vez antes de comenzar a ejecutar todas las pruebas.
- `beforeEach()`: Útil cuando necesitas establecer un estado inicial (como abrir una página específica o preparar datos) antes de cada prueba.
- `after()`: Se suele usar para ejecutar código que limpie o deshaga configuraciones globales después de que todas las pruebas hayan terminado.
- `afterEach()`: Se utiliza para revertir los cambios realizados durante las pruebas, como limpiar el almacenamiento local o borrar cookies después de cada prueba.

Variaciones del bloque it() en Cypress

Variación de it()	Descripción	Uso Común	Ejemplo
<code>it()</code>	Define un test individual que Cypress ejecutará.	Crear pruebas que validen una funcionalidad específica.	<pre>it('cargar la página principal', () => { cy.visit('/'); cy.contains('Bienvenido'); });</pre>
<code>it.only()</code>	Ejecuta solo este test, ignorando los demás dentro de la suite.	Útil para depuración y ejecución focalizada durante el desarrollo de tests.	<pre>it.only('probar esta función específica', () => { cy.get('button').click(); cy.url().should('include', '/dashboard'); });</pre>
<code>it.skip()</code>	Omite la ejecución de este test específico.	Ignorar temporalmente pruebas que no deberían ejecutarse, quizás porque están en desarrollo o bloqueadas por un bug.	<pre>it.skip('probar el login (a corregir)', () => { cy.visit('/login'); cy.get('input[name="user"]').type('admin'); });</pre>
<code>it.todo()</code>	Crea un recordatorio para implementar un test en el futuro.	Marcar pruebas pendientes o que necesitan ser completadas en fases posteriores.	<pre>it.todo('probar la validación del formulario');</pre>

Explicación de las variaciones del bloque it():

- `it()`: Es el bloque básico donde se escribe una prueba. Cada test describe y valida una funcionalidad en particular.
- `it.only()`: Permite ejecutar únicamente ese test. Se usa para enfocar la depuración en una prueba específica sin ejecutar el resto.
- `it.skip()`: Sirve para ignorar temporalmente un test, útil cuando el test no está listo o depende de algo que aún no se ha implementado.
- `it.todo()`: Ayuda a marcar los tests que necesitan ser implementados más adelante, útil para planificar el trabajo.